NAMA: AYU ANGGRAINI
NIM: 24031554054
KELAS: 2024B

JAWABAN SOAL UTS GSD

2.1 Wrangling dan EDA (Maks. skor: 45)

1. **Tentukan 2 periode waktu sebagai filter dari data interaksi. Kombinasi periode yang dipilih tiap mahasiswa**
   Pada tahap ini, data interaksi pengguna dan item dibagi menjadi dua periode waktu, yaitu periode P1 sebagai data *training* dan periode P2 sebagai data *testing*. Pemilihan periode dilakukan dengan menggunakan rentang waktu yang berbeda agar data P1 dapat digunakan untuk proses analisis dan perhitungan kemiripan, sedangkan data P2 digunakan untuk mengevaluasi hasil rekomendasi.
   Periode waktu yang digunakan adalah sebagai berikut.

Periode	Rentang Waktu	Keterangan
P1	3-17 Mei 2015	Data Training
P2	1-18 September 2015	Data Testing

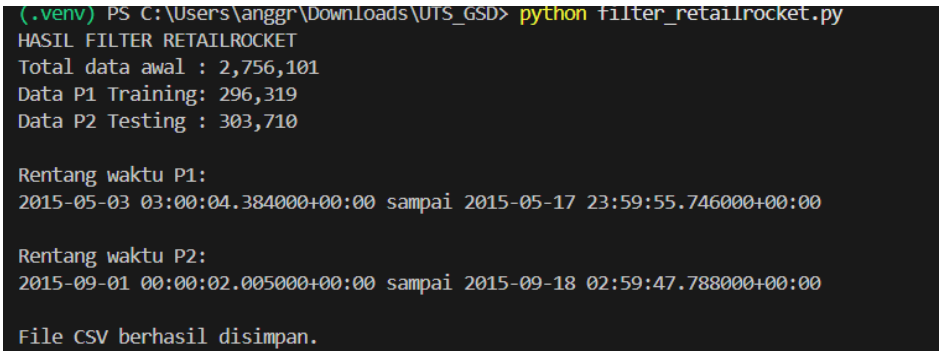
Tabel 1. Periode Waktu yang digunakan

   Berdasarkan pembagian tersebut, periode P1 digunakan sebagai dasar pembentukan matriks interaksi dan perhitungan kemiripan antar pengguna. Sementara itu, periode P2 digunakan untuk mengetahui apakah item yang direkomendasikan sesuai dengan interaksi pengguna pada periode pengujian.

2. **Terapkan proses filter ini pada timestamp pada proses persiapan data, lalu simpan pada sebuah file .csv!**

   Proses filtering dilakukan terhadap kolom timestamp pada dataset RetailRocket untuk memisahkan data interaksi berdasarkan dua periode waktu yang telah ditentukan. Periode P1 digunakan sebagai data *training*, yaitu tanggal 3–17 Mei 2015, sedangkan periode P2 digunakan sebagai data *testing*, yaitu tanggal 1–18 September 2015.

   Hasil filtering disimpan ke dalam dua file CSV, yaitu *events_P1_training.csv* dan *events_P2_testing.csv.* File P1 berisi 296.319 baris data, sedangkan file P2 berisi 303.710 baris data. Kedua file tersebut digunakan sebagai data untuk tahap analisis berikutnya.



Gambar 1. Hasil Filtering Timestamp

3. **Lakukan query untuk mendapatkan komposisi dari subgraph yang dibuat. Uraikan jenis simpul dan jumlah masing-masing simpul dengan Python API! Jelaskan CYPHER SQL yang digunakan!**

Pada tahap ini, dilakukan analisis komposisi subgraph berdasarkan data interaksi pada periode P1 (*training*) dan P2 (*testing*) dari dataset RetailRocket. Graf direpresentasikan menggunakan dua jenis simpul, yaitu Visitor yang merepresentasikan pengguna dan Item yang merepresentasikan produk. Kedua jenis simpul tersebut dihubungkan melalui tiga jenis relasi, yaitu VIEWED untuk aktivitas melihat produk, ADDED_TO_CART untuk aktivitas menambahkan produk ke keranjang, dan BOUGHT untuk aktivitas pembelian.

Hasil perhitungan komposisi subgraph berdasarkan filter timestamp pada masing-masing periode ditunjukkan pada tabel dibawah ini.

Komponen	P1 (Training)	P2 (Testing)
Simpul Visitor	155.916	173.728
Simpul Item	78.171	82.220
Total Simpul	234.087	255.948
Total Relasi Interaksi	296.319	303.710

Tabel 2. Hasil Komposisi Subgraph

Pengambilan komposisi subgraph dilakukan menggunakan query Cypher yang dijalankan melalui Python API dengan Neo4j Driver. Query tersebut digunakan untuk menghitung jumlah simpul unik dan relasi interaksi sesuai dengan periode waktu yang ditentukan.

```
MATCH (v:Visitor)-[r]->(i:Item)
WHERE type(r) IN ['VIEWED', 'ADDED_TO_CART', 'BOUGHT']
RETURN
    count(DISTINCT v) AS jumlah_visitor,
    count(DISTINCT i) AS jumlah_item,
    count(r) AS jumlah_interaksi;
```

Gambar 2. Query Cypher untuk Menghitung Jumlah Visitor, Item, dan Interaksi pada Graf

Parameter $start dan $end digunakan untuk menentukan batas awal dan akhir periode berdasarkan timestamp. Klausa MATCH mencari pola hubungan antara simpul Visitor dan Item, sedangkan WHERE membatasi jenis relasi serta rentang waktu interaksi yang dianalisis. Fungsi count(DISTINCT v) digunakan untuk menghitung jumlah Visitor unik, count(DISTINCT i) untuk menghitung jumlah Item unik, dan count(r) untuk menghitung jumlah relasi interaksi yang memenuhi kondisi query.

Berdasarkan hasil eksekusi program Python, subgraph pada periode P1 memiliki 155.916 simpul Visitor dan 78.171 simpul Item, dengan total 234.087 simpul serta 296.319 relasi interaksi. Sementara itu, subgraph pada periode P2 memiliki 173.728 simpul Visitor dan 82.220 simpul Item, dengan total 255.948 simpul serta 303.710 relasi interaksi.

Hasil tersebut menunjukkan bahwa jumlah simpul dan relasi interaksi pada periode P2 lebih besar dibandingkan periode P1. Hal ini menunjukkan adanya perbedaan komposisi pengguna, produk, dan aktivitas interaksi antara periode training dan testing. Hasil komposisi subgraph ini selanjutnya digunakan sebagai dasar untuk membentuk Vector Space Model (VSM) pada tahap analisis berikutnya.

4. **Lakukan Vector Space Modelling (VSM) terhadap simpul yang diasumsikan sebagai User. Uraikan bagaimana karakteristiknya (dimensi, shape, plot networkx, dst.)!**

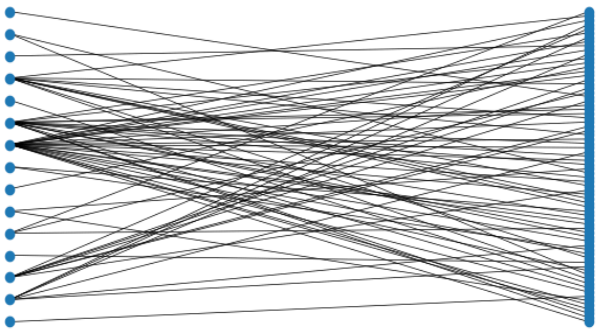
Vector Space Modelling (VSM) dilakukan untuk merepresentasikan aktivitas setiap *Visitor* terhadap *Item* dalam bentuk matriks numerik. Pada tahap ini, setiap baris merepresentasikan satu *Visitor*, sedangkan setiap kolom merepresentasikan satu *Item*. Nilai pada matriks menunjukkan adanya interaksi antara *Visitor* dan *Item* berdasarkan data pada periode P1 (*training*).

Hasil pembentukan VSM Visitor menghasilkan matriks dengan karakteristik sebagai berikut.

Karakteristik	Hasil
Jumlah Visitor	155916
Jumlah Item	78171
Jumlah pasangan unik	231267
Shape matriks	(155916, 78171)
Jumlah dimensi	2
Jumlah elemen matriks	12188109636
Jumlah nilai nonzero	231267
Kerapatan matriks	0,001897%

Tabel 3. Hasil pembentukan *VSM Visitor*

Berdasarkan hasil tersebut, matriks VSM Visitor memiliki 155.916 baris dan 78.171 kolom. Dengan demikian, setiap *Visitor* direpresentasikan oleh vektor yang menunjukkan pola interaksinya terhadap seluruh *Item* pada periode P1. Jumlah elemen tidak nol sebanyak 231.267 menunjukkan bahwa hanya sebagian kecil pasangan *Visitor* dan *Item* yang memiliki interaksi. Hal ini terlihat dari nilai kepadatan matriks yang hanya sebesar 0,001897%, sehingga matriks tergolong sangat *sparse* (jarang terisi). Selanjutnya, matriks VSM digunakan untuk membentuk visualisasi graf menggunakan pustaka NetworkX. Untuk mempermudah pengamatan, visualisasi dilakukan pada sampel yang terdiri atas 15 *Visitor* pertama. Hasil visualisasi sampel tersebut menghasilkan graf dengan **89 simpul dan 74 sisi**. Simpul merepresentasikan *Visitor* dan *Item*, sedangkan sisi menunjukkan adanya interaksi antara keduanya. Visualisasi ini memperlihatkan hubungan antara *Visitor* dan *Item* pada sampel data yang diamati.



Gambar 3 Visualisasi Graf VSM Visitor Menggunakan NetworkX

Berdasarkan hasil VSM Visitor, dapat disimpulkan bahwa data interaksi pada periode P1 memiliki karakteristik yang sangat jarang terisi. Meskipun jumlah *Visitor* dan *Item* cukup besar, tidak semua *Visitor* berinteraksi dengan seluruh *Item*. Representasi dalam bentuk matriks ini dapat digunakan sebagai dasar untuk menganalisis kemiripan pola interaksi antar-*Visitor* dan mendukung proses rekomendasi pada tahap analisis selanjutnya.

5. **Lakukan VSM terhadap simpul yang diasumsikan sebagai Item. Uraikan bagaimana karakteristiknya (dimensi, shape, plot networkx, dst.)!**

Vector Space Modelling (VSM) pada simpul *Item* dilakukan untuk merepresentasikan pola interaksi setiap *Item* terhadap *Visitor* dalam bentuk matriks. Berbeda dengan VSM Visitor, pada VSM Item setiap baris merepresentasikan satu *Item*, sedangkan setiap kolom merepresentasikan satu *Visitor*. Pembentukan matriks menggunakan data interaksi pada periode P1 (*training*).

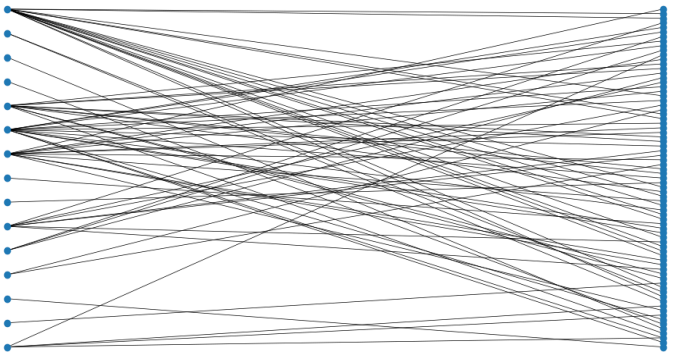
Hasil pembentukan VSM Item memiliki karakteristik sebagai berikut.

Karakteristik	Hasil
Jumlah Item	78171
Jumlah Visitor	155916
Jumlah pasangan unik	231267
Shape matriks	(78171, 155916)
Jumlah dimensi	2
Jumlah elemen matriks	12188109636
Jumlah nilai nonzero	231267
Kerapatan matriks	0,001897%

Tabel 4. Hasil pembentukan VSM Item

Matriks VSM Item memiliki 78.171 baris dan 155.916 kolom. Setiap *Item* direpresentasikan oleh vektor yang menunjukkan pola interaksinya dengan seluruh *Visitor*. Jumlah elemen tidak nol sebanyak 231.267 dengan kepadatan sebesar 0,001897% menunjukkan bahwa matriks sangat *sparse*. Hal ini berarti hanya sebagian kecil pasangan *Item* dan *Visitor* yang memiliki interaksi pada periode P1.

Untuk memahami hubungan antara *Item* dan *Visitor*, dilakukan visualisasi graf menggunakan pustaka NetworkX. Visualisasi dibuat berdasarkan sampel 15 *Item* pertama agar hubungan antarsimpul lebih mudah diamati. Berdasarkan hasil program, graf sampel tersebut terdiri atas **90 simpul dan 76 sisi**. Simpul merepresentasikan *Item* dan *Visitor*, sedangkan sisi menunjukkan adanya interaksi antara keduanya.



Gambar 4. Visualisasi Graf VSM Item Menggunakan NetworkX

Berdasarkan hasil tersebut, VSM Item memberikan representasi pola interaksi setiap *Item* dengan *Visitor*. Matriks yang terbentuk memiliki ukuran besar dan bersifat sangat *sparse*, sehingga tidak semua *Item* memiliki hubungan dengan banyak *Visitor*. Representasi ini dapat digunakan sebagai dasar untuk menganalisis kemiripan antar-*Item* berdasarkan pola interaksi yang dimiliki.

6. **Berdasarkan eksplorasi no. 4 dan 5, uraikan proyeksi manakah yang lebih baik (efektif/efisien)!**

Berdasarkan hasil eksplorasi pada VSM Visitor dan VSM Item, kedua proyeksi memiliki jumlah elemen tidak nol (*non-zero*) yang sama, yaitu 231.267, dengan kepadatan matriks sebesar 0,001897%. Perbedaannya terletak pada dimensi matriks dan orientasi representasi interaksi.

VSM Visitor memiliki *shape* sebesar (155.916, 78.171), sedangkan VSM Item memiliki *shape* sebesar (78.171, 155.916). VSM Visitor merepresentasikan pola interaksi setiap *Visitor* terhadap *Item*, sementara VSM Item merepresentasikan pola interaksi setiap *Item* terhadap *Visitor*. Dengan demikian, keduanya menyimpan informasi interaksi yang sama, tetapi dari sudut pandang yang berbeda.

Dari segi efektivitas, **VSM Visitor lebih sesuai apabila analisis difokuskan pada kemiripan perilaku pengunjung**, misalnya untuk menemukan *Visitor* dengan pola interaksi serupa sebagai dasar sistem rekomendasi. Sebaliknya, VSM Item lebih sesuai apabila tujuan analisis adalah menemukan *Item* yang memiliki pola interaksi serupa.

Dari segi efisiensi, kedua proyeksi memiliki tingkat kepadatan yang sama sehingga tidak terdapat perbedaan sparsitas. Namun, ukuran matriks VSM Visitor lebih besar dari sisi jumlah baris, sedangkan VSM Item memiliki jumlah kolom yang lebih banyak. Oleh karena itu, efisiensi komputasi bergantung pada operasi yang dilakukan dan metode penyimpanan matriks yang digunakan, bukan hanya pada orientasi matriks.

Berdasarkan tujuan analisis yang berfokus pada kemiripan antar-*Visitor* dan rekomendasi *Item* kepada pengunjung, **VSM Visitor lebih tepat dipilih sebagai proyeksi utama**. Proyeksi ini memungkinkan pengukuran kemiripan antar-*Visitor* berdasarkan pola interaksinya, yang selanjutnya dapat dimanfaatkan dalam pendekatan rekomendasi berbasis tetangga terdekat (*K-Nearest Neighbors* atau KNN).

7. **Berdasarkan analisis no. 6, lakukan perhitungan similarity pada hasil graf yg terproyeksi dengan menggunakan Python**
    a. **Tuliskan CYPHER SQL untuk mendapatkan 5 records dengan similarity tertinggi (Anda tentukan sendiri record acuannya)!**

Perhitungan *similarity* dilakukan menggunakan metode *Cosine Similarity* pada matriks VSM Visitor periode P1 (*training*). Perhitungan dilakukan menggunakan Python untuk memperoleh lima Visitor yang memiliki pola interaksi paling mirip dengan Visitor acuan, yaitu Visitor dengan ID 53842. Hasil perhitungan kemudian disimpan pada graf Neo4j sebagai relasi SIMILAR_TO dengan properti similarity.

Untuk menampilkan lima Visitor dengan nilai *similarity* tertinggi, digunakan query Cypher berikut.

```
MATCH (u:Visitor)-[r:SIMILAR_TO]->(v:Visitor)
WHERE u.visitorId = 53842
RETURN
    u.visitorId AS visitor_acuan,
    v.visitorId AS visitor_mirip,
    r.similarity AS similarity
ORDER BY similarity DESC
LIMIT 5;
```

Gambar 5. Query Cypher untuk Menampilkan Lima Visitor Paling Mirip Berdasarkan Nilai Cosine Similarity

Query tersebut mengambil relasi SIMILAR_TO yang berasal dari Visitor acuan dengan ID 53842, menampilkan ID Visitor acuan, ID Visitor yang mirip, dan nilai *similarity*, kemudian mengurutkan hasil berdasarkan nilai *similarity* dari yang tertinggi dan membatasi hasil sebanyak lima baris. Hasil query ditampilkan pada tabel berikut.

No	Visitor Acuan	Visitor Mirip	Similarity
1	53842	710907	0,102763
2	53842	809822	0,093717
3	53842	227091	0,0929074
4	53842	552453	0,085636
5	53842	163561	0,081873

Tabel 5. Hasil Similarity Lima Visitor Teratas

Berdasarkan hasil tersebut, Visitor 710907 memiliki nilai *similarity* tertinggi terhadap Visitor acuan 53842, yaitu 0,102763. Hasil ini menunjukkan bahwa Visitor 710907 merupakan Visitor dengan pola interaksi paling mirip di antara lima hasil teratas.

b. Tuliskan kembali nilai similarity-similarity tersebut pada graf yang telah ada! Perlihatkan hasil query nya!

Setelah nilai *similarity* dihitung menggunakan Python dan disimpan pada graf Neo4j, tahap selanjutnya adalah memvisualisasikan hubungan antara Visitor acuan dan lima Visitor yang memiliki nilai *similarity* tertinggi. Visualisasi dilakukan menggunakan query Cypher berikut.

```
MATCH (u:Visitor)-[r:SIMILAR_TO]->(v:Visitor)
WHERE u.visitorId = 53842
RETURN u, r, v
ORDER BY r.similarity DESC
LIMIT 5;
```

Gambar 6. Query Cypher untuk Visualisasi Hubungan Similarity antara Visitor

Query tersebut digunakan untuk menampilkan simpul Visitor acuan dengan ID 53842, simpul Visitor yang memiliki kemiripan dengannya, serta relasi SIMILAR_TO yang menghubungkan kedua simpul. Hasil diurutkan berdasarkan nilai *similarity* dari yang tertinggi dan dibatasi sebanyak lima relasi

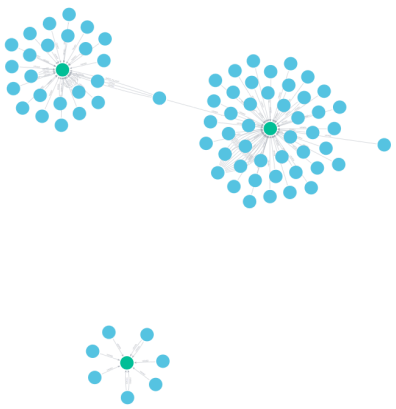


Gambar 7. Visualisasi Relasi Similarity Antar-Visitor pada Neo4j

Berdasarkan visualisasi graf, Visitor dengan ID 53842 berperan sebagai simpul acuan yang terhubung dengan lima Visitor lainnya melalui relasi SIMILAR_TO. Nilai *similarity* yang tersimpan pada setiap relasi menunjukkan tingkat kemiripan pola interaksi antara Visitor acuan dan Visitor lainnya. Visualisasi ini membantu memperlihatkan hasil perhitungan kemiripan dalam bentuk graf sehingga hubungan antar-Visitor dapat diamati secara langsung.

8. **Tuliskan CYPHER SQL dan tunjukkan hasil graf-nya untuk memperlihatkan kondisi: sebelum dan sesudah perhitungan similarity!**
   a. Kondisi sebelum perhitungan similarity

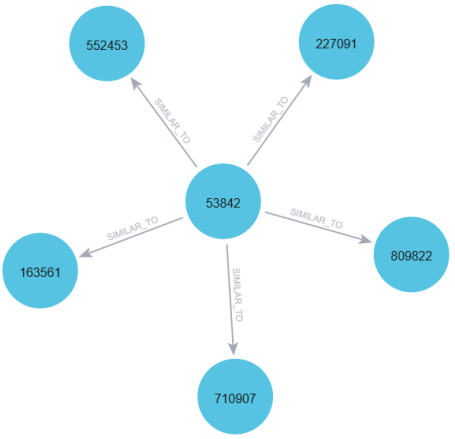
   Sebelum perhitungan *similarity*, graf Neo4j merepresentasikan hubungan interaksi antara Visitor dan Item melalui relasi VIEWED, ADDED_TO_CART, dan BOUGHT. Hubungan tersebut menggambarkan aktivitas Visitor terhadap Item berdasarkan data interaksi pada periode yang dianalisis.



Gambar 8. Graf Interaksi Visitor dan Item Sebelum Perhitungan Similarity

b. Kondisi sesudah perhitungan similarity

Setelah perhitungan *Cosine Similarity*, hasil kemiripan disimpan ke dalam graf Neo4j menggunakan relasi berarah SIMILAR_TO dengan properti similarity. Relasi ini menghubungkan Visitor acuan dengan lima Visitor yang memiliki pola interaksi paling mirip berdasarkan hasil perhitungan.



Gambar 9. Graf Relasi Similarity Antar-Visitor Setelah Perhitungan Similarity

Berdasarkan perbandingan kedua graf, sebelum perhitungan *similarity*, hubungan yang ditampilkan berfokus pada interaksi antara Visitor dan Item. Setelah perhitungan dilakukan, graf dapat menampilkan hubungan kemiripan antar-Visitor melalui relasi SIMILAR_TO. Dengan demikian, hasil perhitungan Python dapat dimanfaatkan untuk memperkaya representasi hubungan pada Neo4j dan mendukung proses rekomendasi berbasis kemiripan pola interaksi.

2.2 Analytics (Maks. skor: 55)

1. **Pilih seorang user secara acak (tentukan proses acak-nya, dan harus bisa di-reproduksi), lalu**

Pemilihan user dilakukan menggunakan Python dengan menetapkan *random seed* sebesar 42 agar proses pemilihan dapat direproduksi. Berdasarkan hasil eksekusi program, user yang terpilih adalah Visitor dengan ID 1095818. User tersebut

digunakan sebagai acuan untuk menghasilkan rekomendasi Item berdasarkan kemiripan pola interaksi dengan user lainnya pada periode P1 (*training*).

a. tuliskan skrip Python dalam merekomendasikan KNN-nya!

Metode *K-Nearest Neighbors* (KNN) digunakan untuk mencari lima Visitor yang memiliki pola interaksi paling mirip dengan user acuan. Kemiripan dihitung menggunakan *Cosine Similarity* berdasarkan matriks VSM Visitor. Selanjutnya, interaksi dari lima tetangga terdekat digunakan untuk menghasilkan daftar Item yang direkomendasikan.

Implementasi lengkap program tersebut terdapat pada file **knn_recommendation.py**. Program dijalankan melalui terminal menggunakan perintah berikut.: *python knn_recommendation.py*

Hasil eksekusi program digunakan untuk memperoleh lima tetangga terdekat dan tiga Item rekomendasi sesuai dengan kebutuhan soal.

b. berikan 3 item yang diiklankan untuk k = 5 pada penentuan KNN nya!

Berdasarkan hasil eksekusi program, diperoleh lima tetangga terdekat dan nilai kemiripannya sebagai berikut.

No	ID Visitor Tetangga	Cosine Similarity
1	527797	1,000000
2	400691	1,000000
3	1084964	1,000000
4	323427	0,707107
5	1365440	0,577350

Tabel 6. Hasil Penentuan Lima Tetangga Terdekat (KNN) untuk User 1095818

Lima tetangga tersebut digunakan untuk menghasilkan rekomendasi Item bagi user dengan ID 1095818. Sesuai permintaan soal, tiga Item rekomendasi teratas ditampilkan pada tabel berikut.

Peringkat	ID Item Rekomendasi
1	80131
2	344603

3	273326

Tabel 7. Tiga Item Rekomendasi Teratas untuk User 1095818

Berdasarkan hasil tersebut, sistem KNN menghasilkan tiga rekomendasi Item teratas, yaitu Item 80131, 344603, dan 273326. Rekomendasi diperoleh berdasarkan pola interaksi dari lima Visitor terdekat terhadap user acuan. Dengan demikian, proses KNN menghasilkan daftar Item yang dapat direkomendasikan kepada user berdasarkan kemiripan pola interaksi.

2. Berdasarkan hasil rekomendasi Anda no. 1b Analytics, hitunglah:

a. Hit@5

Berdasarkan hasil eksekusi program Python, diperoleh satu Item relevan pada periode P2, yaitu Item ID 300755. Namun, Item tersebut tidak ditemukan dalam lima rekomendasi teratas, yaitu Item ID 80131, 344603, 273326, 287090, dan 116444.

Rumus Hit@5:

$$Hit@5 \ = 0 \ jika \ tidak \ ada \ item \ relevan \ di \ Top \ - \ 5$$

$$Hit@5 \ = 1 \ jika \ ada \ item \ relevan \ di \ Top \ - \ 5$$

Karena tidak ada Item relevan dalam lima rekomendasi teratas, diperoleh:

$$Hit@5 \ = \ 0$$

b. NDCG@5

NDCG@5 digunakan untuk mengukur kualitas urutan rekomendasi berdasarkan relevansi Item.

Rumus:

$$NDCG@5 \ = \ DCG@5/IDCG@5$$

Karena Item relevan 300755 tidak muncul dalam lima rekomendasi teratas, nilai $DCG@5 = 0$. Dengan satu Item relevan pada data pengujian, nilai $IDCG@5 = 1$. Maka:

$$NDCG@5 \ = \ 0/1 \ = \ 0,0000$$

Kesimpulan: Hasil evaluasi menunjukkan Hit@5 sebesar 0 dan NDCG@5 sebesar 0,0000. Pada pengujian terhadap user 1095818, sistem belum berhasil memasukkan Item relevan ke dalam lima rekomendasi teratas.